# Federated Concept-Based Models: Interpretable models with distributed supervision

**Anonymous Author(s)**
Affiliation
Address
`email`

## Abstract

Concept-based Models (CMs) enhance interpretability in deep learning by grounding predictions in human-understandable concepts. However, concept annotations are costly and rarely available at scale within a single data source. Federated Learning (FL) could alleviate this limitation by enabling cross-institutional training over concept annotations distributed across multiple data owners. Yet, FL lacks interpretable modeling paradigms. Integrating CMs with FL is non-trivial: although FL supports heterogeneous and non-stationary client participation, it typically assumes a fixed shared architecture, whereas CMs may require architectural adaptation as the available concept set evolves. We propose *Federated Concept-based Models* (F-CMs), a new methodology for deploying CMs in evolving FL settings. F-CMs aggregate concept-level information across institutions and efficiently adapt the model architecture to changes in concept supervision while preserving privacy. Empirically, F-CMs maintain accuracy and intervention effectiveness comparable to training settings with full concept supervision, while outperforming on average non-adaptive federated baselines. Notably, F-CMs enable interpretable inference on concepts unavailable to a given institution, a key novelty over existing approaches.

## 1 Introduction

In many real-world applications, the deployment of machine learning systems is subject to constraints beyond predictive accuracy: practitioners need models whose predictions are interpretable and auditable, providing tools for error diagnosis, fairness assessment, and compliance with legal standards [1–3]. Among existing approaches, *Concept-based Models* (CMs) offer an established framework in which predictions are formulated in terms of high-level, human-interpretable variables, referred to as *concepts* [4–6]. In practice, however, their applicability is constrained by the need for explicit concept supervision during training [7, 8]. Concept annotations are often expensive to obtain, time-consuming, and rarely available at scale within a single data source.

A natural way to alleviate this limitation is to pool concept knowledge across multiple data owners, such as hospitals, research centers, or devices [9, 10]. *Federated Learning* (FL) provides a principled paradigm for collaborative training without sharing raw data, making it particularly appealing in privacy-sensitive settings where concept annotations themselves may encode sensitive information [11]. However, combining CMs with FL creates a fundamental mismatch: FL is designed for dynamic settings in which clients may join or leave and client data distributions may differ, while still relying on a shared model architecture with a fixed parameterization. CMs challenge this assumption, as their architecture is explicitly tied to the available concept set and, often, to the dependencies among concepts. As a result, changes that FL can typically absorb at the level of client participation or data distribution may require architectural adaptation in CMs. This issue becomes especially pronounced in realistic FL deployments, where several sources of variation may exist [12–15].

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

To bridge this gap, we introduce *Federated Concept-based Models (F-CMs)*, a new methodology for deploying concept-based architectures in realistic federated settings. Crucially, F-CMs handle *statistical heterogeneity* and *temporal non-stationarity*: as new clients join (potentially from new data distributions), they may introduce previously unseen concepts and novel dependencies between them. To accommodate this, F-CMs expand the shared concept space and, when available, update the concept dependency structure. Furthermore, the architecture is adapted by updating only the affected components, thereby avoiding full retraining while preserving previously learned knowledge. To the best of our knowledge, F-CMs are the first to address evolving concept spaces under temporal non-stationarity (dynamic participation and data drift) in FL by adapting the model structure, instead of assuming a fixed shared model. Our key contributions benefit both communities and are as follows:

- We introduce **F-CMs**, a novel methodology for deploying **concept-based architectures in realistic federated settings**, addressing heterogeneous, partial concept supervision and temporal non-stationarity.
  - For evolving FL settings, it provides a systematic way to incorporate architectural **interpretability** while allowing the shared model structure to adapt during training, moving beyond the fixed-architecture assumption of standard FL.
  - For interpretability, it **reduces the cost** for concept annotations at a single institution by enabling privacy-preserving knowledge sharing.
- We instantiate F-CMs with four CM architectures and validate them on synthetic benchmarks and two real-world medical imaging datasets. Results show **comparable predictive and intervention accuracy** w.r.t. full concept supervision, enable **interpretable inference over concepts missing at individual clients**, and **reduce retraining cost** under federation growth.

With this work, we hope to contribute toward the broader goal of scaling interpretable models while remaining grounded in real-world constraints and deployment scenarios.

# 2 Preliminaries

## 2.1 Concept-based Models (CMs)

CMs predict a task through human-interpretable concepts. In what follows, we consider two main families of CMs, which we refer to as *bipartite CMs* and *graph-based CMs*.

**Bipartite CMs**, exemplified by [5, 6], map an input $x \in \mathcal{X}$ to a latent representation $z = g(x) \in \mathcal{Z}$, where $z$ is a learned, generally non-interpretable representation of the input. This representation is used to predict a set of $m$ concepts as $\hat{c}_j = h_j(z)$, where each $\hat{c}_j$ may be scalar- or vector-valued. The full concept vector is then given by $\hat{\mathbf{c}} = (\hat{c}_1, \ldots, \hat{c}_m) \in \mathcal{C}$. The task prediction is computed from the concepts and, possibly, the latent input representation $\hat{y} = f(\hat{\mathbf{c}}, z) \in \mathcal{Y}$. This model induces a *directed acyclic graph* (DAG) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ [16] over interpretable variables $\mathcal{V} = \{C_1, \ldots, C_m, Y\}$, where edges $\mathcal{E}$ are directed only from concepts to the task, i.e., $C_j \to Y$ for all $j$.

**Graph-based CMs**, exemplified by [17, 18], instead allow for dependencies among concepts. In this case, concept and task follow a more general DAG $\mathcal{G}$, where each concept may depend on a set of parent concepts. Concretely, concept and task predictions can be written as $\hat{c}_j = h_j(\hat{\mathbf{c}}_{\mathrm{PA}(C_j)}, z)$ and $\hat{y} = f(\hat{\mathbf{c}}_{\mathrm{PA}(Y)}, z)$, where PA denotes the parent set induced by $\mathcal{G}$. Bipartite CMs can be recovered as a special case of graph-based CMs with only concept-to-task edges.

## 2.2 Federated Learning (FL)

FL trains a shared model across multiple data owners (*clients*) without sharing raw data [11]. Let $m_{\mathbf{w}} : \mathcal{X} \to \mathcal{Y}$ denote the shared model with parameters $\mathbf{w}$. The client population may vary over time, denoted by $\mathcal{K}(t)$; at round $t$, the server selects a subset $\mathcal{K}_t \subseteq \mathcal{K}(t)$ for participation [13, 19]. Each client $k \in \mathcal{K}(t)$ holds a private dataset $\mathcal{D}^{(k)} = \{(x_i^{(k)}, y_i^{(k)})\}_{i=1}^{n^{(k)}}$. In standard FL, the goal is to minimize a weighted global objective $F(\mathbf{w}) = \sum_{k=1}^{K} p^{(k)} F^{(k)}(\mathbf{w})$, where $F^{(k)}(\mathbf{w}) = \mathbb{E}_{(x,y)\sim\mathcal{D}^{(k)}}[\ell(m_{\mathbf{w}}(x), y)]$ is the local objective of client $k$ and $p^{(k)}$ typically weighs clients proportionally to their data size. At round $t$, the server broadcasts the current weights $\mathbf{w}_t$ to clients in $\mathcal{K}_t$, each selected client performs local optimization on $\mathcal{D}^{(k)}$, and the server aggregates the returned updates to obtain $\mathbf{w}_{t+1}^{(k)}$, typically via *FedAvg* [11], i.e., by weighted averaging proportional to local dataset sizes.

**Statistical heterogeneity and temporal non-stationarity.** Practical FL deployments are rarely stationary [20]. First, FL is typically *statistically heterogeneous*: different clients observe different data distributions, i.e., $\exists\, k \neq k'$ s.t. $\mathbb{P}^{(k)}(X,Y) \neq \mathbb{P}^{(k')}(X,Y)$, so their local objectives $F^{(k)}$ differ [14, 21]. Second, FL can be *temporally non-stationary*, meaning that the data and/or the set of participating clients changes over rounds. This commonly arises through (i) *dynamic client participation*, where the available client pool $\mathcal{K}(t)$ and sampled participants $\mathcal{K}_t$ vary over time as clients join or leave [19]; and (ii) *data drift*, where even for a fixed client $k$, its distribution changes over time, i.e., $\mathbb{P}_t^{(k)}(X,Y) \neq \mathbb{P}_{t'}^{(k)}(X,Y)$ for $t \neq t'$ [15, 22]. Both effects induce a time-varying optimization landscape and can destabilize global updates.

These conditions violate key assumptions of standard CMs. Usually, standard CMs assume a concept set that determines the architecture. In FL settings, however, concept annotations would be distributed across clients and may evolve with clients or data, so concepts and their dependencies can change over time. Consequently, a federated CM must (i) reconcile heterogeneous, partial concept supervision across clients, and (ii) dynamically update concepts, their dependencies and corresponding architecture during training, rather than relying on a fixed design upfront.

### 2.3 Problem Setting

We study FL of CMs under statistical heterogeneity and temporal non-stationarity. Training proceeds over rounds $t=1,\dots,T$, where a subset of clients $\mathcal{K}_t$ participates. Client $k$ holds a local dataset $\mathcal{D}^{(k)} = \{(x_i^{(k)}, \mathbf{v}_i^{(k)})\}_{i=1}^{n^{(k)}}$, where $x_i^{(k)} \in \mathcal{X}$ is the input and $\mathbf{v}_i^{(k)}$ contains the available supervision for sample $i$, i.e., concept annotations and/or task label. We denote by $\mathcal{V}^{(k)} \neq \emptyset$ the whole set of interpretable variables (concepts and task) supervised by client $k$. Each client $k$ additionally provides a DAG $\mathcal{G}^{(k)}$, represented by a *weighted adjacency matrix* $A^{(k)} \in [0,1]^{|\mathcal{V}^{(k)}|\times|\mathcal{V}^{(k)}|}$ [23]. Notably, for bipartite CMs, the structure is fixed a priori by allowing only concepts-to-task edges.

Clients may dynamically join or leave the federation over time; when a client leaves, its raw data is discarded, while its contribution to the learned model is retained. Consequently, the global set of supervised variables evolves as $\mathcal{V}_t = \bigcup_{\tau=1}^{t} \bigcup_{k\in\mathcal{K}_\tau} \mathcal{V}^{(k)}$.

The problem is to learn, at each round $t$, a shared CM $F_{\mathbf{w}_t}$ that minimizes the federated loss induced by the supervision available at each client, while adapting architecture as the global variable set $\mathcal{V}_t$ and its dependencies evolve. Each client contributes only losses for the variables it supervises, so the server must reconcile partial objectives without accessing raw data.

## 3 Federated Concept-based Models

We now formalize **Federated Concept-based Models (F-CMs)** as a federated training framework for bipartite and graph-based CMs under partial, heterogeneous concept supervision and temporal non-stationarity; Algorithm 1 in App. E.1 summarizes the full procedure. As in standard FL, F-CMs maintain a single shared CM optimized through repeated rounds of client-side training and server-side aggregation. Beyond standard FL, F-CMs support an evolving concept space via three key extensions:

1. **Graph aggregation**: at each round t, the server aggregates client-provided structural information ($A^{(k)}$) into a shared DAG $\mathcal{G}_t$ encoding dependencies and defining the model architecture;
2. **Dynamic architecture adaptation**: the shared CM is decomposed into *concept-specific* and *task-specific* modules, where each module denotes the component used to predict a given concept or the task label. This enables the interpretable architecture to expand and rewire modularly as new concepts or dependencies emerge. For instance, when a client introduces a new concept or a new concept dependency, F-CMs add or modify only the modules associated with the involved concepts, while leaving the remaining components unchanged.
3. **Module-specific optimization**: each client updates only the modules associated with the interpretable variables it supervises, while modules for unobserved concepts or task labels remain frozen. Server-side aggregation is then performed separately for each module, using only the updates from clients that provided supervision for the corresponding variable.

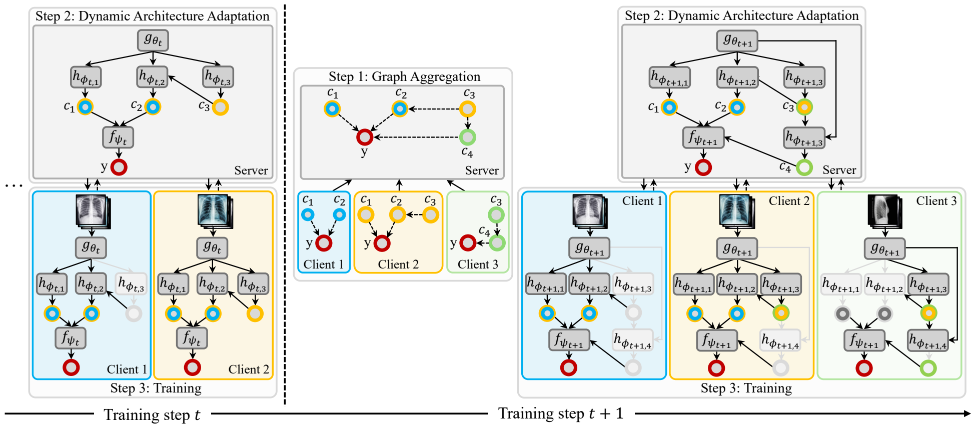


Figure 1: **F-CMs overview.** At each round $t$, F-CMs performs three steps: (1) *Graph aggregation:* the server combines client graphs into a shared graph $\mathcal{G}_t$; (2) *Architecture adaptation:* the shared CM adapts to $\mathcal{G}_t$ and/or newly observed concepts; (3) *Training:* clients update only supervised concept/task modules (dark grey), leaving others frozen (light grey), and the server aggregates the corresponding updates. The figure shows two rounds, where new clients introduce new concepts.

### 3.1 F-CM Shared Model

At round $t$, the server maintains a shared CM (bipartite or graph-based), with parameters $\mathbf{w}_t = (\theta_t, \phi_t, \psi_t)$, which maps inputs $x$ to predictions over variables $\mathcal{V}_t$ through the following modules:

- a *latent encoder* $g_{\theta_t} : \mathcal{X} \to \mathbb{R}^d$ producing $z = g_{\theta_t}(x)$;
- a collection of *concept encoders* $\{h_{\phi_t,j}\}_j$ predicting the value of concepts $C_j \in \mathcal{V}_t \setminus \{Y\}$ as $\hat{c}_j = h_{\phi_t,j}\big(\hat{\mathbf{c}}_{\mathrm{PA}_t(C_j)}, z\big)$;
- a *task decoder* $f_{\psi_t}$ producing $\hat{y} = f_{\psi_t}\big(\hat{\mathbf{c}}_{\mathrm{PA}_t(Y)}, z\big)$;

Here, $\mathrm{PA}_t$ denotes the parent set in a shared DAG $\mathcal{G}_t$ constructed over $\mathcal{V}_t$ by aggregating client-specific DAGs $\mathcal{G}^{(k)}$, as described in Section 3.2. In bipartite instantiations, $\mathcal{G}_t$ allows only concept-to-task edges, hence $\mathrm{PA}_t(C_j) = \emptyset$ for every concept $C_j$, and $\mathrm{PA}_t(Y) = \mathcal{V}_t \setminus \{Y\}$.

**Instantiations.** F-CMs are agnostic to the specific CM architecture used as the shared model. An instantiation specifies the parameterization of modules $\{h_{\phi_t,j}\}_j$ and $f_{\psi_t}$.

### 3.2 F-CM Pipeline

We next detail how F-CMs aggregate structure, adapt the shared architecture, and perform module-specific optimization under partial supervision. Figure 1 provides an overview.

#### 3.2.1 Graph Aggregation

A single client's structural knowledge can be unreliable: local graphs may be noisy due to limited, non-IID data and may also be manipulated by compromised clients (i.e., poisoning) [24]. We therefore aggregate client-provided structures into a shared DAG $\mathcal{G}_t$ over the current global variable set $\mathcal{V}_t$, by favoring edges and directions *with the strongest support across observed clients*.

Specifically, each client $k$ sends a representation of its local DAG $\mathcal{G}^{(k)}$ to the server only upon joining or when its local structure changes, encoded as a weighted adjacency matrix $A^{(k)}$ over its supervised variables $\mathcal{V}^{(k)}$. We interpret $A^{(k)}_{ij}$ as the client's confidence in the directed edge $V_i \to V_j$. For each unordered pair $\{V_i, V_j\} \subseteq \mathcal{V}_t$ with $i \neq j$, we define a categorical distribution over the three possible outcomes $\{V_i \to V_j,\ V_j \to V_i,\ \varnothing\}$, with probabilities $A^{(k)}_{ij}, A^{(k)}_{ji}, A^{(k)}_{\varnothing,ij}$, respectively. Here, $A^{(k)}_{\varnothing,ij} := 1 - A^{(k)}_{ij} - A^{(k)}_{ji}$ denotes the confidence assigned by client $k$ to the absence of an edge between $V_i$ and $V_j$. The server aggregates client confidences into a *strength* matrix by summing contributions from all clients observed up to round $t$, denoted by $\mathcal{K}_{\leq t}$, using the latest graph received from each client:

$$[\bar{A}_t]_{ij} := \sum_{k\in\mathcal{K}_{\le t}:\, V_i, V_j\in\mathcal{V}^{(k)}} \alpha_k\, A_{ij}^{(k)},$$

where $\alpha_k$ are aggregation weights (e.g., proportional to client dataset sizes, or uniform/reliability-aware), and $[\bar{A}_t]_{ij}=0$ if the sum is empty. The same rule is applied to $[\bar{A}_t]_{\varnothing,ij}$. The shared graph $\mathcal{G}_t$ is then constructed by applying a maximum-consensus rule to each pair

$$(V_i\to V_j)\in\mathcal{G}_t \iff [\bar{A}_t]_{ij} = \max\big\{[\bar{A}_t]_{ij}, [\bar{A}_t]_{ji}, [\bar{A}_t]_{\varnothing,ij}\big\}.$$

Ties are broken at random. Cycles are resolved iteratively (Sec. C.2).

### 3.2.2 Dynamic Architecture Adaptation

The shared DAG $\mathcal{G}_t$ from the previous step defines the CM connectivity by specifying predicted variables and dependencies. Under non-stationarity, these may change as new clients introduce unseen concepts or revised dependencies. Accordingly, F-CMs adapt the shared CM only when module interfaces change, while preserving all unaffected ones. At round $t$, this amounts to updating the concept encoders $\{h_{\phi_t,j}\}_j$ and $f_{\psi_t}$. The latent encoder $g_{\theta_t}$ remains unchanged.

**(E1) Adding a new concept.** For any newly observed concept $C_j$, the server instantiates a new module $h_{\phi_t,j}$. The task decoder is updated if $C_j\in\mathrm{PA}_t(Y)$, while other modules remain unchanged.

**(E2) Updating an edge (add/remove/re-orient).** In graph-based F-CMs, changing the relation between two variables modifies parent sets and thus module inputs: adding $C_i\to C_j$ appends $\hat{c}_i$ to the inputs of $h_{\phi,j}$; removing $C_i\to C_j$ removes it; re-orienting $C_i\to C_j$ into $C_j\to C_i$ is treated as remove+add, so $h_{\phi,j}$ loses $\hat{c}_i$ while $h_{\phi,i}$ gains $\hat{c}_j$. The task decoder is updated analogously. In bipartite F-CMs, updates are constrained, disallowing concept-to-concept edges and re-orientations.

**Warm-start initialization.** When the module input expands, only the newly introduced parameters are initialized, while existing parameters are preserved. For example, weights connected to new inputs may be zero-initialized to preserve pre-update behavior, or initialized randomly. Modules whose interfaces are unchanged retain their parameters exactly, avoiding full retraining.

### 3.2.3 Module-Specific Training and Module-Wise Aggregation

Because concept and task supervision are unevenly distributed across clients, vanilla FedAvg [11] would mix updates for modules that some clients cannot supervise. F-CMs therefore perform module-specific training and module-wise aggregation, restricting contributions to supervised components. This induces a masked federated optimization problem, where each client optimizes a client-specific partial objective over a subset of shared parameters.

**Module-specific local optimization.** At round $t$, each client $k\in\mathcal{K}_t$ receives the broadcasted shared CM $(\mathbf{w}_t,\mathcal{G}_t)$, where $\mathbf{w}_t$ denotes the pre-training parameters obtained after architecture adaptation, and performs $E$ local optimization steps using only its available supervision. The local objective is

$$F^{(k)}(\mathbf{w}_t;\mathcal{G}_t) = \mathbb{E}_{(x,\mathbf{v})\sim\mathcal{D}^{(k)}}\Bigg[\gamma \sum_{C_j\in\mathcal{V}^{(k)}\setminus\{Y\}} \ell_c\big(\hat{c}_j, c_j\big) + (1-\gamma)\,\mathbf{1}_{\{Y\in\mathcal{V}^{(k)}\}}\,\ell_t\big(\hat{y}, y\big)\Bigg], \qquad (1)$$

where $\ell_c$ and $\ell_t$ are standard concept and task losses (e.g., cross-entropy), and $\gamma\in[0,1]$. Only supervised modules contribute gradients; unsupervised ones are frozen and need not be transmitted.

**Module-wise aggregation.** The server aggregates each module using only the clients that produced updates for that module. We use a FedAvg-like rule, although F-CMs can in principle support other aggregation rules. Let $\mathcal{K}_t^j\subseteq\mathcal{K}_t$ denote the clients updating concept module $h_{\phi_t,j}$, yielding local parameters $\phi_{t+1,j}^{(k)}$. The aggregated parameters are

$$\phi_{t+1,j} = \sum_{k\in\mathcal{K}_t^j} \beta_j^{(k)}\,\phi_{t+1,j}^{(k)}, \qquad \beta_j^{(k)} = \frac{n^{(k)}}{\sum_{r\in\mathcal{K}_t^j} n^{(r)}}.$$

The task module is aggregated analogously over clients that updated it, while modules not updated by any client remain unchanged. Since all participating clients update the latent encoder, its parameters are aggregated with the same FedAvg-like rule over $\mathcal{K}_t$.

Table 1: **Task accuracy** (%) and **concept coverage** (%) with respect to the ground-truth concepts causally relevant for the task, i.e., concepts with a directed path to the task in the ground-truth graph or the available proxy. Methods that improve the average performance are shown in bold. Results are reported as mean ± standard error over five runs.

| Setting | Method | Asia | | Sachs | | Alarm | | Insurance | | Hailfinder | | SIIM-Pn. | | CheXpert | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | T | C. Cov. | T | C. Cov. | T | C. Cov. | T | C. Cov. | T | C. Cov. | T | C. Cov. | T | C. Cov. |
| Cent. (Upper Bound) | OpaqNN | 80.7±1.0 | 100 | 77.7±1.0 | 100 | 74.0±0.3 | 100 | 76.3±1.1 | 100 | 73.0±0.8 | 100 | 74.1±0.6 | 100 | 72.0±0.3 | 100 |
| | CBM | 80.0±1.0 | 100 | 77.3±0.9 | 100 | 73.9±0.5 | 100 | 76.3±1.4 | 100 | 74.2±0.7 | 100 | 73.9±0.3 | 100 | 70.7±0.5 | 100 |
| | CEM | 80.4±1.1 | 100 | 77.3±0.8 | 100 | 71.6±0.4 | 100 | 76.0±1.2 | 100 | 73.3±0.6 | 100 | 73.1±0.7 | 100 | 66.6±0.7 | 100 |
| | CGM | 79.8±1.2 | 100 | 76.2±1.2 | 100 | 71.5±0.9 | 100 | 73.4±1.1 | 100 | 72.2±0.8 | 100 | 68.8±0.8 | 100 | 71.6±0.3 | 100 |
| | C$^2$BM | 80.3±1.0 | 100 | 77.9±1.0 | 100 | 72.8±0.5 | 100 | 74.8±0.9 | 100 | 73.5±0.6 | 100 | 72.8±0.1 | 100 | 69.4±0.7 | 100 |
| Loc. | OpaqNN | 56.5±6.7 | 46.7±3.3 | 50.2±9.4 | 52.0±4.9 | 45.0±11.1 | 67.0±5.1 | 19.4±6.2 | 72.6±6.1 | 48.1±7.9 | 63.7±3.5 | 50.9±8.6 | 59.4±3.1 | 57.7±4.7 | 31.4±5.4 |
| | CBM | 47.7±3.8 | 46.7±3.3 | 50.1±7.0 | 52.0±4.9 | 51.7±7.0 | 67.0±5.1 | 30.9±7.6 | 72.6±6.1 | 46.6±7.6 | 63.7±3.5 | 44.2±6.2 | 59.4±3.1 | 57.7±4.2 | 31.4±5.4 |
| | CEM | 67.1±7.4 | 46.7±3.3 | 52.2±8.8 | 52.0±4.9 | 42.1±11.1 | 67.0±5.1 | 40.8±5.3 | 72.6±6.1 | 44.1±7.1 | 63.7±3.5 | 62.4±1.4 | 59.4±3.1 | 57.3±4.7 | 31.4±5.4 |
| | CGM | 64.7±5.1 | 46.7±3.3 | 37.0±9.4 | 52.0±4.9 | 46.1±8.9 | 50.4±6.5 | 31.7±7.2 | 70.5±7.4 | 44.1±7.3 | 63.7±3.5 | 56.6±6.0 | 59.4±3.1 | 60.9±4.9 | 31.4±5.4 |
| | C$^2$BM | 66.7±7.1 | 46.7±3.3 | 37.3±13.8 | 52.0±4.9 | 45.0±8.9 | 50.4±6.5 | 22.1±12.1 | 70.5±7.4 | 43.0±5.6 | 63.7±3.5 | 53.8±6.2 | 59.4±3.1 | 58.6±4.2 | 31.4±5.4 |
| External Static FL | FCL | 63.9±6.4 | N/A | 59.5±1.3 | N/A | 50.1±5.9 | N/A | 62.3±1.8 | N/A | 31.7±2.6 | N/A | 56.5±5.9 | N/A | 50.9±0.9 | N/A |
| | FedCBM | 68.0±4.8 | 48.6±5.7 | 48.7±9.4 | 54.0±5.1 | 54.8±8.8 | 70.0±4.1 | 59.7±8.7 | 76.9±2.4 | 51.1±7.4 | 84.4±2.9 | 54.6±4.6 | 63.6±3.7 | 66.1±4.0 | 34.3±3.5 |
| S-F-CMs | OpaqNN | 74.0±4.9 | 46.7±3.3 | 55.2±9.1 | 56.0±4.0 | 61.7±7.4 | 71.3±5.4 | 64.3±8.5 | 83.2±2.6 | 54.0±7.6 | 84.7±2.8 | 63.1±1.1 | 68.8±3.6 | 56.1±9.0 | 34.3±3.5 |
| | CBM | 63.5±3.6 | 46.7±3.3 | 42.2±12.1 | 56.0±4.0 | 57.0±8.0 | 71.3±5.4 | 58.4±11.8 | 83.2±2.6 | 51.8±10.8 | 84.7±2.8 | 55.8±6.2 | 68.8±3.6 | 67.0±4.2 | 34.3±3.5 |
| | CEM | 72.8±8.0 | 46.7±3.3 | 39.9±15.2 | 56.0±4.0 | 56.2±10.9 | 71.3±5.4 | 67.0±3.4 | 83.2±2.6 | 55.1±9.7 | 84.7±2.8 | 56.8±6.5 | 68.8±3.6 | 54.9±9.2 | 34.3±3.5 |
| | CGM | 70.8±8.3 | 46.7±3.3 | 39.3±14.9 | 56.0±4.0 | 54.6±10.5 | 51.3±6.7 | 58.2±11.8 | 76.8±5.9 | 50.6±7.7 | 83.3±2.4 | 61.8±0.3 | 68.8±3.6 | 65.7±3.8 | 34.3±3.5 |
| | C$^2$BM | 70.4±7.7 | 46.7±3.3 | 46.5±15.5 | 56.0±4.0 | 57.1±10.4 | 51.3±6.7 | 60.1±13.9 | 78.9±7.1 | 59.1±8.2 | 83.3±2.4 | 50.0±8.0 | 68.8±3.6 | 61.6±4.9 | 34.3±3.5 |
| **F-CMs** | OpaqNN | **80.5±1.1** | **100** | **75.2±1.0** | **100** | **72.7±0.7** | **100** | **71.3±0.6** | **100** | **68.0±1.4** | **100** | **64.8±1.0** | **100** | **70.0 ±0.2** | **100** |
| | CBM | **80.1±0.9** | **100** | **75.2±0.9** | **100** | **71.0±1.9** | **100** | **69.5±0.6** | **100** | **70.6±1.5** | **100** | **67.6±1.1** | **100** | **69.2 ±0.7** | **100** |
| | CEM | **80.7±1.0** | **100** | **75.7±1.0** | **100** | **72.8±0.7** | **100** | **71.0±0.5** | **100** | **71.6±1.0** | **100** | **68.0±0.1** | **100** | **67.0 ±0.1** | **100** |
| | CGM | **77.8±1.3** | **100** | **71.7±1.8** | **100** | **68.5±3.3** | **100** | **70.0±1.1** | **100** | **63.8±3.4** | **100** | **63.6±0.2** | **100** | **67.5 ±0.3** | **100** |
| | C$^2$BM | **80.4±1.0** | **100** | **76.2±1.0** | **100** | **73.3±0.5** | **100** | **74.2±1.1** | **100** | **72.0±1.8** | **100** | **65.6±2.0** | **100** | **66.1 ±1.3** | **100** |

**Theoretical analysis.** F-CM training is non-standard: clients may have partial supervision, aggregation is module-wise, and the architecture may change. Appendix E.2 provides a theoretical analysis for a static version of F-CM with fixed clients and concept supervision, allowing us to study, in isolation, the effects of partial supervision and module-wise aggregation on training. Specifically, we show that if a global objective exists under standard assumptions, and the aggregated module-wise updates are sufficiently aligned with its gradient, then the usual nonconvex stationarity behavior is recovered up to a bounded mismatch. Exact alignment gives the standard $O(T^{-1/2})$ rate. For F-CMs, this intuition applies piecewise over intervals with fixed clients and supervision.

# 4 Experimental Evaluation

In our experiments, we instantiate F-CMs with two bipartite CMs, CBM [5] and CEM [6], and two graph-based CMs, CGM [17] and C$^2$BM [18]. CBM predicts scalar concepts, CEM predicts concept embeddings, while CGM and C$^2$BM condition concept predictions on parent concepts in a DAG (details in App. B). Specifically, C$^2$BM assumes a *causal* DAG, where each directed edge represents a direct causal relation between variables. By contrast, CGMs can be instantiated with different types of graphs. To provide a unified treatment across graph-based models, we use causal DAGs also for CGMs. This is an implementation choice: F-CMs can also be used with other DAG structures.

We empirically evaluate F-CMs across interpretability and FL objectives. Specifically, we assess (i) task accuracy and concept coverage (Section 4.1), (ii) responsiveness to ground-truth concept interventions (4.2), and (iii) training efficiency under evolving federations with dynamic architecture updates (4.3). The appendix complements this evaluation with extended intervention and adaptation analyses across architectures and instantiations, and analyses of the extent and computational overhead of modular adaptation (D.1–D.5). We also report privacy–utility trade-offs under client-level differential privacy (D.6), sensitivity analyses with respect to the number of clients, selective parameter freezing, and the concept-loss weight (D.7), robustness to inter-client semantic disagreement and to noisy client graphs (D.8 and D.9), and an ablation of graph aggregation strategies under graph perturbations (D.10). Finally, we study a natural extension of F-CMs to multimodal settings, particularly relevant in real-world FL where clients access heterogeneous modalities while sharing common semantic targets; App. D.11 formalizes this extension and reports an experiment on a realistic multimodal dataset.

**Baselines and training regimes.** We compare F-CM instantiations to an opaque neural baseline (**OpaqNN**) under four training regimes: (i) **Centralized (Cent.)**, pooling all client data and concept annotations, serving as an upper bound; (ii) **F-CMs** (ours); (iii) **S-F-CMs**, a static federated variant

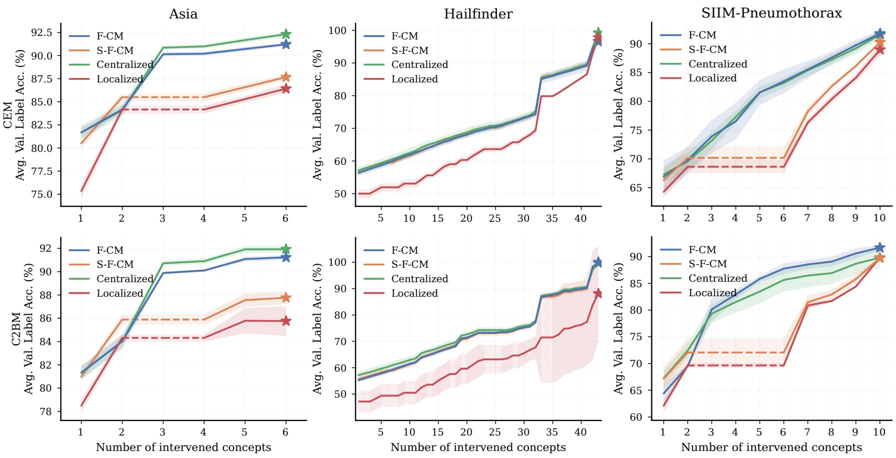


Figure 2: **Label accuracy** (%) on downstream variables (including the task) following interventions on concepts at deeper levels of the ground-truth graph hierarchy. Dashed lines denote interventions that are **not** possible for a given modality due to the absence of predictions for the concept.

of F-CMs that uses the same optimization and structure aggregation, but fixes the concept space and architecture after the first-round client participation (no adaptation); (iv) **Localized (Loc.)**, training independent client models without sharing. We also include two state-of-the-art federated concept-learning baselines, FedCBM [25] and FCL [26] reported separately as **external static FL baselines** since they assume a static federated setting and rely on method-specific concept-learning pipelines.

**Federated protocol under temporal non-stationarity.** Federated experiments are conducted under *non-stationarity* induced by dynamic client participation and evolving concept supervision. At each round $t$, the server samples $|\mathcal{K}_t| = 10$ clients. The client pool $\mathcal{K}(t)$ evolves up to 20 clients during training: after a warm-up phase (at $t = 10$ for ASIA, SACHS, and SIIM, and at $t = 20$ otherwise; App. C.1), new clients join with additional concept supervision and, for graph-based F-CMs, potentially new dependencies. Experiments up to $|\mathcal{K}_t| = 100$ are reported in App. D.7.1

**Datasets and data preparation under heterogeneous partial concept supervision.** We consider five Bayesian network benchmarks from `bnlearn` [27] (ASIA[28], SACHS [29], INSURANCE [30], ALARM [31], and HAILFINDER [32]), each with a ground-truth DAG, and two real-world chest X-ray datasets: SIIM-PNEUMOTHORAX [33], with concept annotations generated using a medical CLIP model following established practices [34, 18], and CHEXPERT [35], whose labels are extracted from radiology reports. For each dataset, one variable is treated as the task and the remaining as concepts. To model heterogeneous settings, clients are assigned different data distributions and observe partial subsets of concepts. In the centralized setting, graph-based CMs use the ground-truth DAG or a proxy, whereas in federated and localized ones clients access only their local subgraphs and concepts. To model imperfect knowledge (or data poisoning), client graphs are perturbed by randomly adding, removing, or reversing edges (see App. A and C.1).When computing accuracy, variables that a model cannot predict under a given training regime (e.g., localized) are assigned chance-level accuracy, defined as the expected accuracy of a uniform random predictor over the corresponding space.

### 4.1 Task and Concept Coverage

We evaluate *task accuracy* and *concept coverage*, i.e., the fraction of task-relevant concepts predicted by the model. *Concept accuracy* is reported in App. D.1.

**Federated aggregation is necessary under statistical heterogeneity.** Comparing S-F-CMs to Localized isolates the effect of cross-client aggregation when the concept space is fixed. Across benchmarks, S-F-CMs improve task accuracy and coverage over localized training, showing that aggregating supervision and, for graph-based models, structure is essential to reconcile client heterogeneity.

**Aggregation alone is insufficient under temporal non-stationarity; F-CMs close the gap.** Under temporal non-stationarity, late-arriving clients introduce additional supervision and, for graph-based

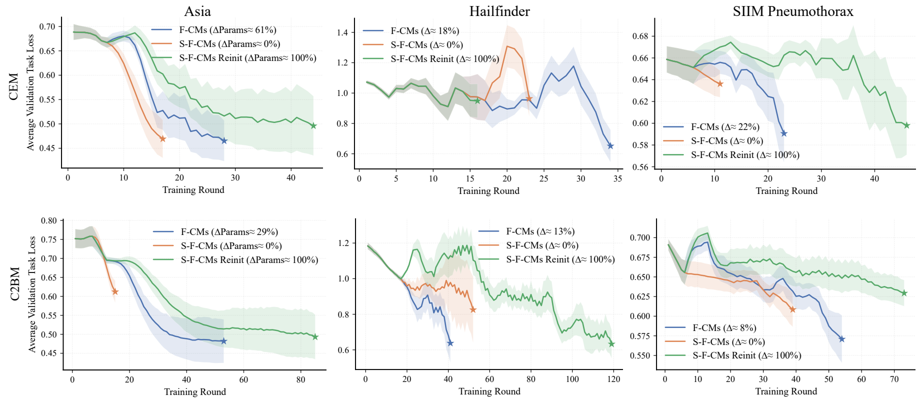


Figure 3: **Convergence under temporal non-stationarity.** Average task loss across rounds for C$^2$BM and CEM, where new clients introduce unseen concepts. F-CMs (ours) adapt the architecture and converge faster and to lower loss than S-F-CMs (no adaptation) and S-F-CMs Reinit (retraining). Curves are truncated at the best loss-round. The legend reports the fraction of modified parameters.

models, new dependencies, making a fixed architecture increasingly mismatched to the evolving concept space. This is reflected by the gap between S-F-CMs and both Centralized and F-CMs, and is consistent with the weaker task accuracy of the external static FL baselines FedCBM and FCL. By extending the global concept set and updating the dependency graph, F-CMs adapt the shared architecture and remain close to the centralized bound in task accuracy, while attaining near-100% coverage of task-relevant concepts.

Overall, these results show that realistic federations require (i) graph aggregation to handle heterogeneity and (ii) architecture adaptation to sustain performances as the federation evolves.

### 4.2 Concept-Level Interventions

In this section, we evaluate responsiveness to *ground-truth concept interventions* by replacing an increasing number of predicted concepts with their ground-truth values, mimicking human corrections at deployment. To ensure a consistent policy, interventions are sampled within progressively deeper levels in the ground-truth DAG hierarchy. Fig. 2 reports the average accuracy over the remaining predicted variables (concepts and task) after each intervention, which we refer to as *label accuracy*. For clearer visualization we fix the client subgraphs across three seeds, isolating intervention effects from variability in the federated partition. Additional results are reported in App. D

**F-CMs enable interventions on locally unannotated concepts.** Dashed segments mark interventions that are impossible because the corresponding concepts are not predicted by a given regime. This is the case for both Localized and S-F-CMs: localized models cannot predict concepts outside a client's subgraph, while static federation cannot incorporate concepts introduced by late clients because it fixes the concept space early. By combining cross-client aggregation with dynamic expansion and rewiring, F-CMs predict the broadest set of concepts and keep benefiting from further interventions, yielding higher label accuracy.The same trend holds for task accuracy alone (App. D.2.1).[1]

**F-CMs achieve interventional responsiveness comparable to centralized.** Across datasets and architectures, F-CMs achieve accuracy comparable to centralized setting throughout the intervention trajectory, showing that F-CMs preserves gains from concept corrections as pooled training does.

### 4.3 Efficient Adaptation in Non-Stationary Federations

We evaluate how F-CMs behave in *non-stationary* federations where new clients join or leave during training and introduce previously unseen concepts, requiring the shared model to expand. Fig.3 reports the average validation task loss over communication rounds on the same datasets and instantiations

[1]The Localized regime shows higher variance because, with limited concept supervision, the model may encode information about unobserved concepts into the predicted ones; interventions overwrite these predictions, leading to less stable gains.

of the previous experiment. We compare: **F-CMs** (ours), **S-F-CMs** (without architecture adaptation), and **S-F-CMs Reinit** (re-training from scratch whenever new clients join).

**Across all datasets and both architectures, F-CMs converge faster and to lower task loss.** F-CMs outperform S-F-CMs and avoid the slow recovery of the reinitialization baseline, which erases knowledge accumulated from earlier clients. This translates into greater *adaptation efficiency* in evolving federations: F-CMs modify only the modules affected by newly introduced concepts or dependencies, yielding sparse parameter changes (e.g., $8\%$–$61\%$ across settings), compared to $0\%$ for S-F-CMs (which cannot incorporate new concepts) and $100\%$ for S-F-CMs Reinit. Full results on parameter changes and additional plots for the remaining instantiations are reported in App. D.4 and D.3. These results highlight that modular architecture adaptation yields a more convergence-efficient strategy for continual federation growth, preserving previously learned knowledge while integrating new supervision.

## 5 Related works

We build on the literature on concept-based models (CMs), which leverage human-interpretable variables for prediction and interpretation. In our experiments, we consider representative architectures from this line of work, including CBM [5], CEM [6], CGM [17], and C$^2$BM [18]. Several other CM variants have also been proposed, e.g., [8, 36, 37]. Our framework is not tied to the architectures evaluated here and could be extended to accommodate other CM variants.

A related line of work addresses the challenge of obtaining concept annotations. Specifically, several CM extensions address the challenge of obtaining concept annotations by relaxing the need for fully observed concept supervision, e.g., by handling missing or noisy labels and combining supervised with unsupervised concept representations [34, 38–41]. However, they typically rely on domain-specific pre-trained models to infer concepts. In contrast, our approach leverages ground-truth concept annotations, even when distributed across multiple entities.

Interpretability in FL remains relatively underexplored. Recent works [25, 26, 42] take initial steps toward combining concepts with FL, but rely on unsupervised concept extraction [43, 44] rather than concept annotations. Consequently, they cannot fully exploit FL to aggregate existing concept-level supervision across clients. They also assume a *static* federation, and therefore do not address *temporal non-stationarity*, common in real-world FL, where new clients may introduce previously unseen concepts and dependencies, requiring the shared concept space and architecture to expand over time.

A related direction is *federated causal discovery*, which studies how to infer causal relationships from decentralized data [45–47]. These methods typically assume a fixed, or pre-specified union, variable set across clients, which is restrictive in settings with evolving concept sets. Moreover, our goal is a model-agnostic graph aggregation mechanism rather than a method tied specifically to causal discovery. Beyond this, *graph federated learning* [48–51] focuses on training graph neural networks via federated optimization, rather than learning evolving interpretable concept-based architectures.

## 6 Conclusions

We introduced F-CMs, a novel methodology for training concept-based models in realistic federated settings under heterogeneous, partial concept supervision and temporal non-stationarity (dynamic participation and data drift). F-CMs aggregate concept supervision and, when available, concept dependencies across clients throughout training, while adapting the shared model modularly as the concept space evolves. Experiments show that F-CMs preserve task performance while improving concept coverage and intervention effectiveness over local and fixed federated architecture baselines. Importantly, F-CMs enable interpretable inference even for concepts that are locally unannotated.

**Limitations and future work.** While F-CMs can reuse existing CM architectures, these models were not designed for evolving concept spaces. A promising direction is to develop federation-aware CMs with greater modularity, so that concept and dependency updates affect fewer parameters and enable more efficient adaptation under temporal non-stationarity. Another important direction is to strengthen the privacy analysis by systematically characterizing information leakage. Although bringing interpretable models into FL is important, it may also introduce new attack surfaces that remain largely unexplored, motivating defenses tailored to concept-based federated models.